Explainable Cardiovascular Disease Classification

Implementation details and results

Project Review 2

This file goes through what we ran for the project and what came out of it. The numbers and charts all come from the outputs folder of our repository.

1. Overview

The project is about predicting whether a person in the Kaggle cardiovascular disease dataset has a recorded CVD label. The dataset has 70,000 patients and 11 measurements for each, such as age, blood pressure, cholesterol and weight. We trained four models on it: a dummy baseline, logistic regression, random forest and XGBoost. We tuned each one using five-fold cross-validation on the training data, picked the best algorithm using a separate validation set, and only then looked at the test set, and only for that one model. After that we used SHAP to see what the model pays attention to.

We also took the winning algorithm with its settings and trained new models on two other heart datasets, UCI Cleveland and Framingham. These are separate experiments. The columns and the outcome are different in each dataset, so the primary model could not simply be applied to them.

One point to keep in mind throughout: on the primary data the label is something already recorded, so this is classification and not a forecast of who will get sick later.

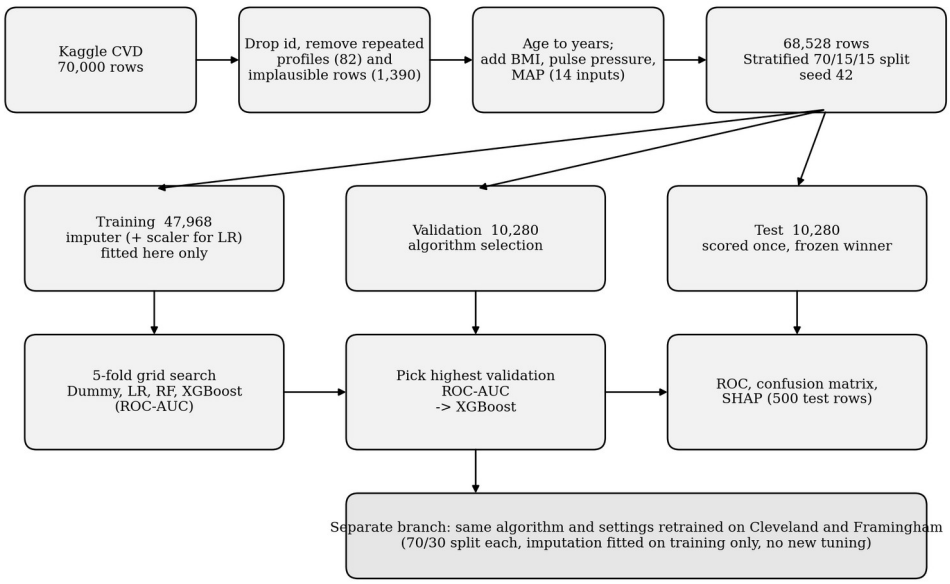


Figure 1. The pipeline as we ran it (scripts 01 to 04).

2. Datasets and exploration

Table 1. Datasets used

Dataset	Rows	Target	Role
Kaggle Cardiovascular Disease	70,000	CVD absent / present	Primary experiment
UCI Cleveland	303	Heart disease absent / present	Separate retraining
Framingham	4,240	Ten-year CHD	Separate retraining

Before cleaning anything we plotted the raw primary data with 00_visualize_data.py. The raw file has some blood pressure readings in the thousands, so the blood pressure plot is clipped to make it readable, and age is rounded to whole years in these plots. None of this clipping or rounding is used when the models are trained.

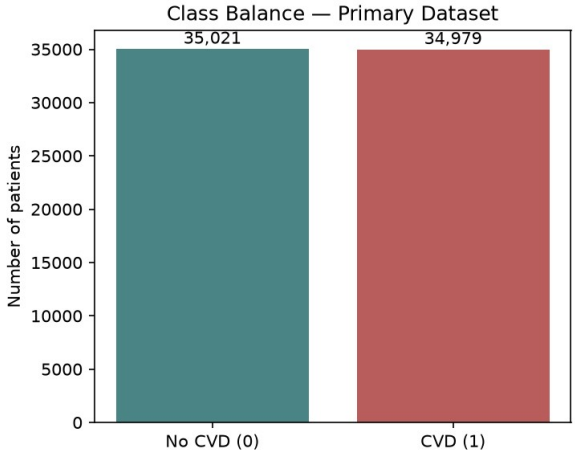


Figure 2. Class balance in the raw data (35,021 without CVD, 34,979 with CVD).

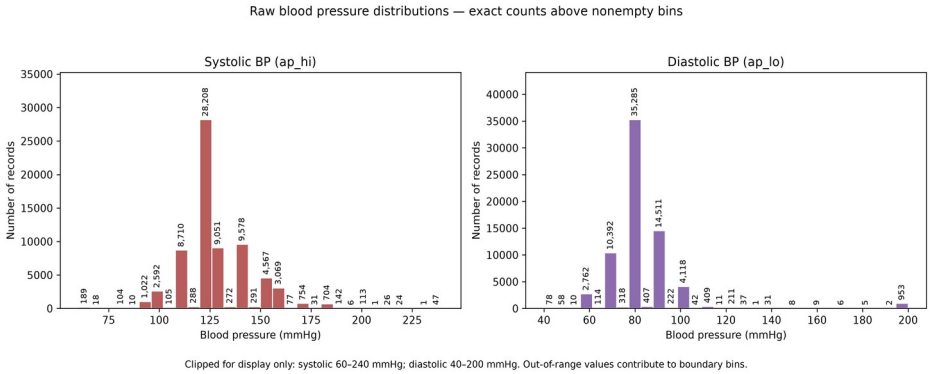


Figure 3. Systolic and diastolic blood pressure, clipped for display.

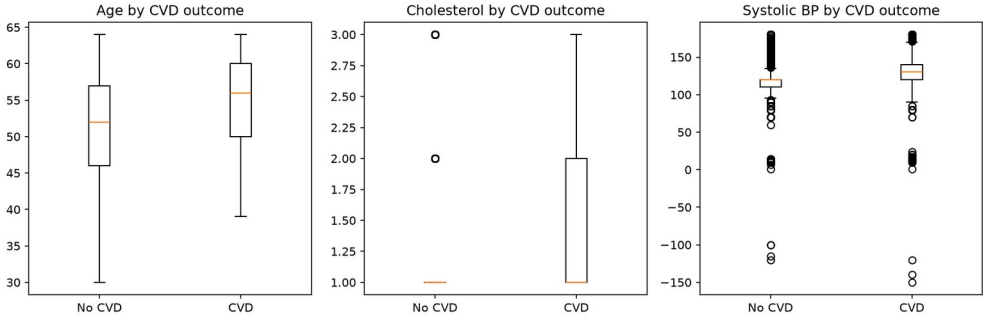


Figure 4. Age, cholesterol and systolic pressure for each outcome.

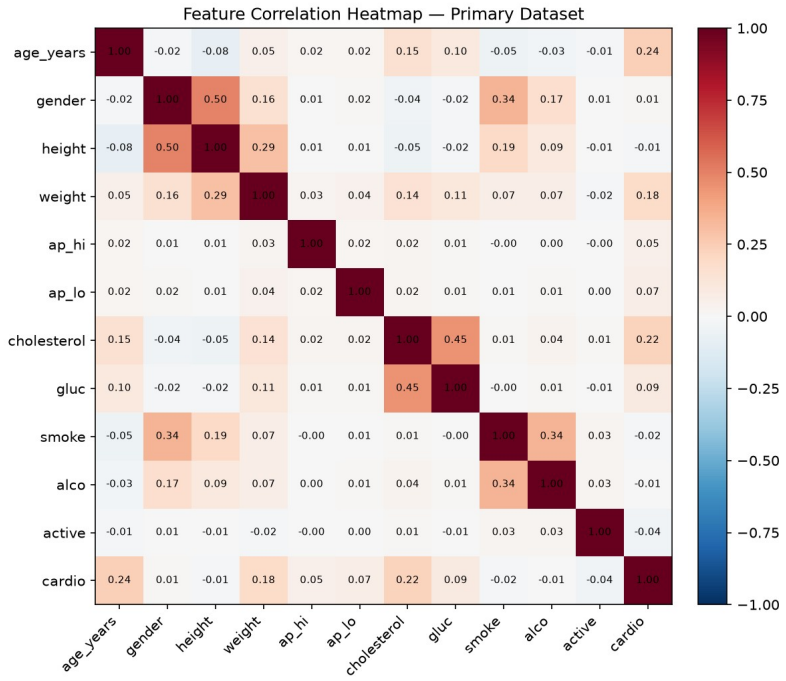


Figure 5. Correlation between the raw features and the target.

3. Preprocessing and splitting

01_preprocess_primary.py does the cleaning. The steps, in the order they run, are:

- Drop the id column. It is just a row number, so it cannot tell us whether a patient appears twice.
- Remove every row that shares all 11 original predictor values with another row. This took out 82 rows. We removed all copies and did not keep one, because 17 of these repeated profiles had different labels and we had no good reason to pick one.
- Remove 1,390 rows with impossible values: systolic pressure outside 60 to 240, diastolic outside 40 to 200, systolic lower than diastolic, height outside 100 to 220 cm, or weight outside 30 to 200 kg.
- Convert age from days to years by dividing by 365.25. We did not round it.
- Add BMI, pulse pressure and mean arterial pressure. With these the model has 14 inputs.

Table 2. Number of records after each step (preprocessing_audit.json)

Stage	Records	Note
Raw	70,000	35,021 no CVD, 34,979 CVD
After removing repeated profiles	69,918	82 removed
After removing impossible values	68,528	1,390 removed
Final, no CVD	34,618	50.52%
Final, CVD	33,910	49.48%

The cleaned data was then split into training, validation and test sets with two stratified train_test_split calls using seed 42. The script also checks that no predictor profile shows up in more than one of the three sets, and it stops if one does. The classes are nearly balanced, so we did no resampling.

Table 3. Primary split

Set	Records	No CVD	CVD
Training	47,968	24,232	23,736
Validation	10,280	5,193	5,087
Test	10,280	5,193	5,087

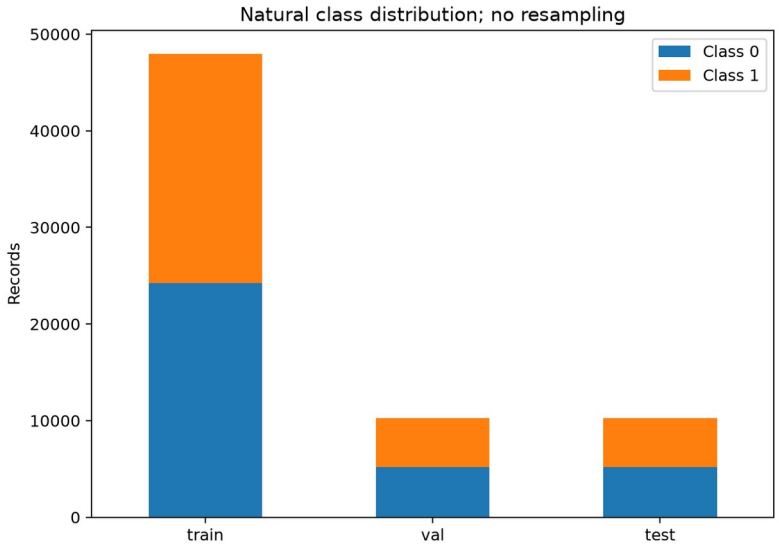


Figure 6. Class counts in each set. The natural distribution was kept.

4. Training and model selection

In 02_train_and_tune.py every model is a scikit-learn pipeline made of a median imputer, a scaler (only for logistic regression) and the model itself. We kept the imputer and scaler inside the pipeline on purpose. That way cross-validation fits them again on each fold, and nothing from the held-out fold gets into them. Each model was tuned with a grid search over shuffled, stratified five-fold cross-validation, scored on ROC-AUC.

Table 4. Grids searched and the settings that were chosen

Model	Grid	Chosen
Dummy	none (predicts the class prior)	n/a
Logistic Regression	C: 0.01, 0.1, 1, 10	C = 0.01
Random Forest	trees 100/200/300; depth 5/10/15	300 trees, depth 10
XGBoost	learning rate 0.01/0.05/0.1; trees 100/200/300; depth 3/5/7	0.1, 100 trees, depth 3

Table 5. Cross-validation and validation results (validation_results.csv)

Model	CV ROC-AUC mean	CV SD	Validation ROC-AUC	Val. accuracy	Val. recall
Dummy	0.5000	0.0000	0.5000	50.52%	0.00%
Logistic Regression	0.7916	0.0041	0.7925	72.71%	66.64%
Random Forest	0.7994	0.0051	0.8003	73.22%	67.23%
XGBoost	0.8005	0.0051	0.8025	73.44%	68.08%

XGBoost had the highest validation ROC-AUC, so it was selected. The file selection.json records that the test set was not used for this choice. Its lead over random forest is only 0.0022, which is smaller than the spread between folds, so we think those two are about equal in practice. The CV scores come from the same folds that were used to choose the settings, so they are tuning estimates and not nested cross-validation results.

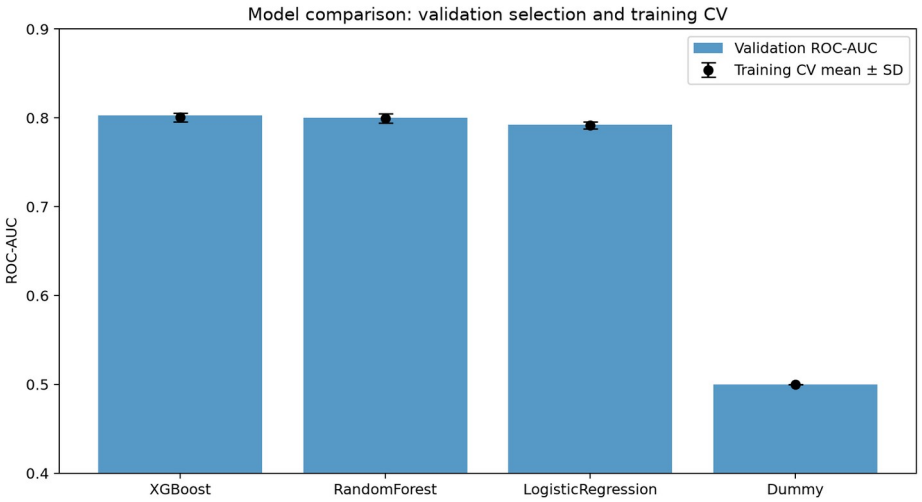


Figure 7. Validation ROC-AUC (bars) next to the training CV mean and SD (points).

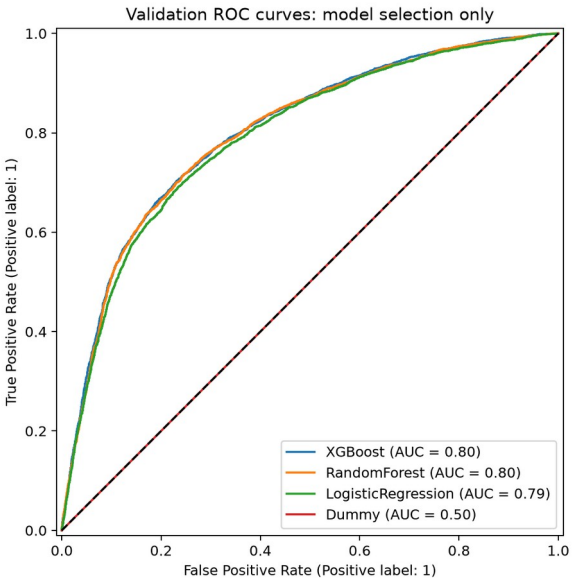


Figure 8. Validation ROC curves. These were used for selection only.

5. Test results

03_evaluate_test_shap.py is the only script that scores the test set. We left the classification threshold at 0.5 and did not tune it.

Table 6. Selected XGBoost model on the primary test set (test_results.csv)

Metric	Value
ROC-AUC	0.8028
Accuracy	73.42%
Precision	75.82%
Recall	67.98%
F1 score	0.7168

The confusion matrix shows 4,090 true negatives, 1,103 false positives, 1,629 false negatives and 3,458 true positives. So at this threshold the model misses roughly one in three of the people who have CVD.

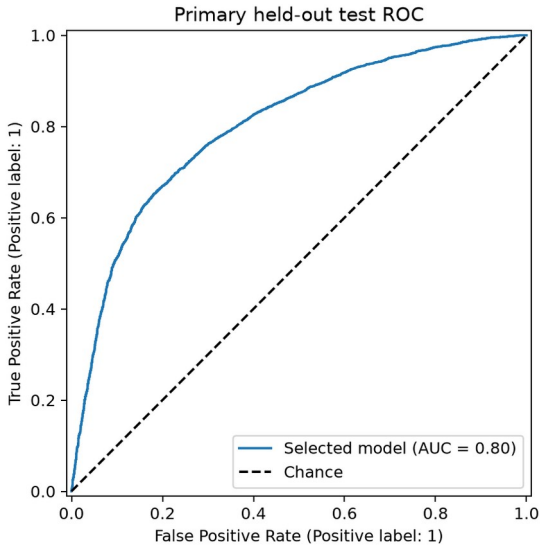


Figure 9. ROC curve on the primary test set.

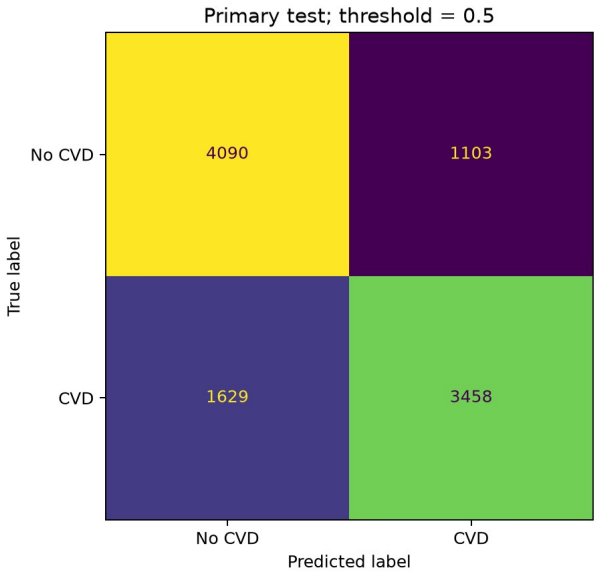


Figure 10. Confusion matrix on the primary test set at threshold 0.5.

6. SHAP explanations

We ran SHAP on 500 randomly chosen test rows (seed 42). For XGBoost we did not use shap.TreeExplainer. The versions of SHAP and XGBoost we had installed did not work together on that path, so the script asks the XGBoost booster for exact TreeSHAP values directly with pred_contribs=True. It then checks that each row's contributions add up to the model's raw output. The values are in log-odds.

Table 7. Mean absolute SHAP value for each feature (shap_importance.csv)

Feature	Mean absolute SHAP
ap_hi (systolic BP)	0.668
age	0.276
map	0.230
cholesterol	0.190
active	0.054
bmi	0.052
weight	0.051
gluc	0.024
smoke	0.023
alco	0.020
pulse_pressure	0.014
ap_lo	0.011
height	0.011
gender	0.009

Systolic blood pressure is well ahead of everything else, with age, mean arterial pressure and cholesterol behind it. Systolic pressure, diastolic pressure, MAP and pulse pressure are all calculated from each other, so SHAP splits the credit among them. We read these numbers as a description of how the model behaves. They do not show what causes CVD.

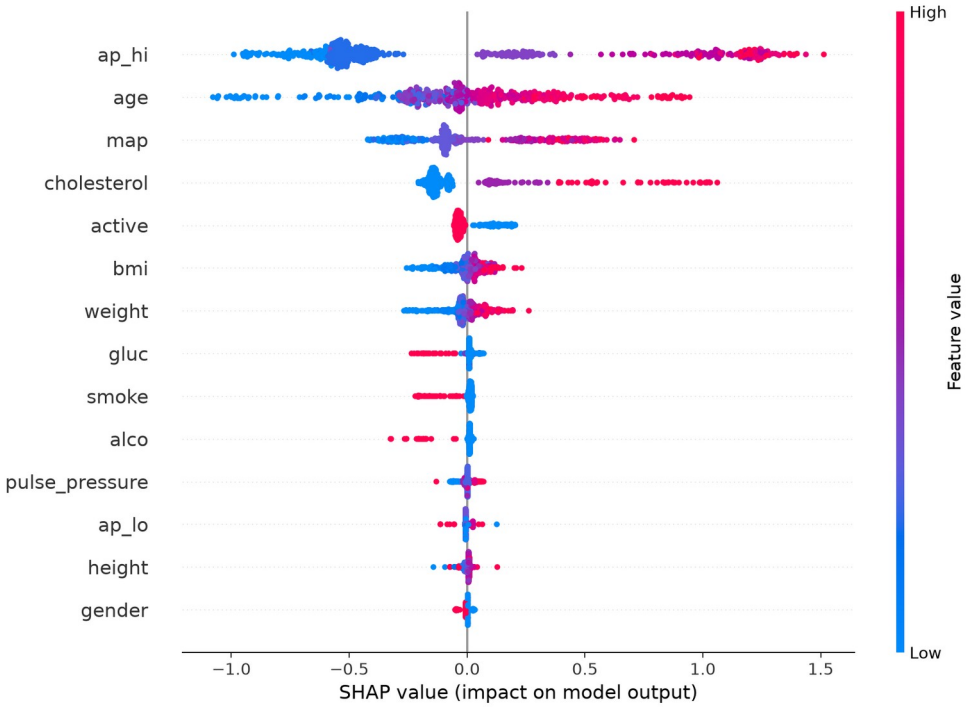


Figure 11. SHAP beeswarm plot for 500 test rows.

Figure 12 explains one test patient (row 4209). The model gave this patient a 14.4% probability of CVD, and the recorded label is no CVD.

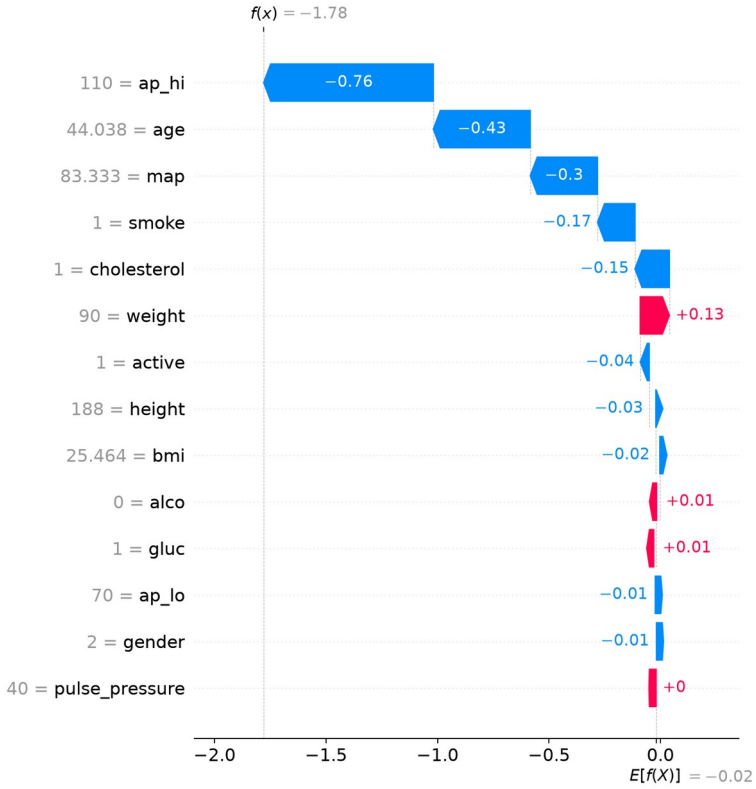


Figure 12. SHAP waterfall plot for one test patient.

7. Retraining on Cleveland and Framingham

04_generalization_test.py reads the winning algorithm and its parameters from validation_results.csv and trains a new model on each dataset. Each one uses a 70/30 stratified split with seed 42, and the imputer is fitted on the training part only. We did no new tuning. A dummy model was run next to each one. No rows were removed as repeats in either dataset.

Table 8. Held-out results (generalization_results.csv and external_audit.json)

Dataset	Model	Train / test	ROC-AUC	Accuracy	Recall	CV ROC-AUC
Cleveland	Dummy	212 / 91	0.5000	53.85%	0.00%	0.5000
Cleveland	XGBoost	212 / 91	0.9052	82.42%	80.95%	0.8821
Framingham	Dummy	2,968 / 1,272	0.5000	84.83%	0.00%	0.5000
Framingham	XGBoost	2,968 / 1,272	0.6816	84.98%	9.33%	0.7096

The Framingham result needs some explanation. Only about 15% of its records are positive, so the dummy model already reaches 84.8% accuracy and our model’s 85.0% is hardly better.

Its recall is 9.3%, meaning it finds fewer than one in ten of the positive cases at a threshold of 0.5. The Cleveland numbers look good, but the holdout there has only 91 people, so they could easily move with a different split.

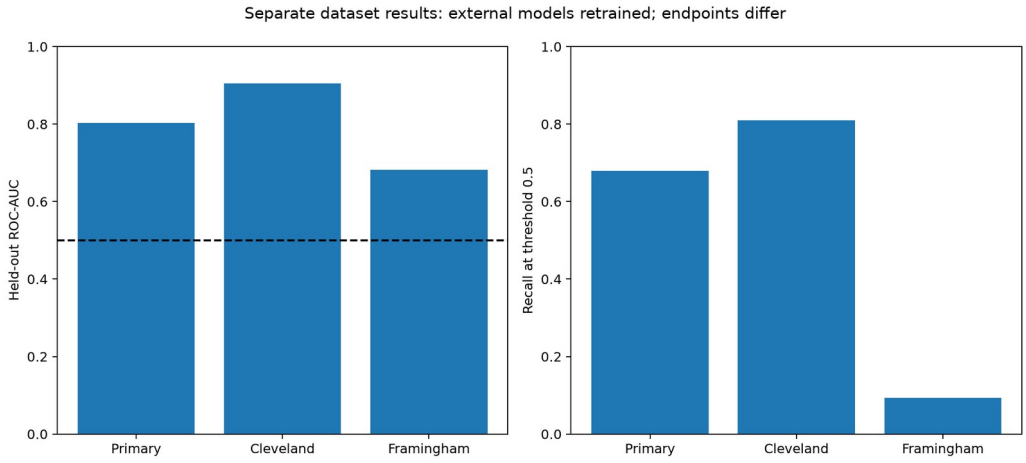


Figure 13. Held-out ROC-AUC and recall for the three separate experiments.

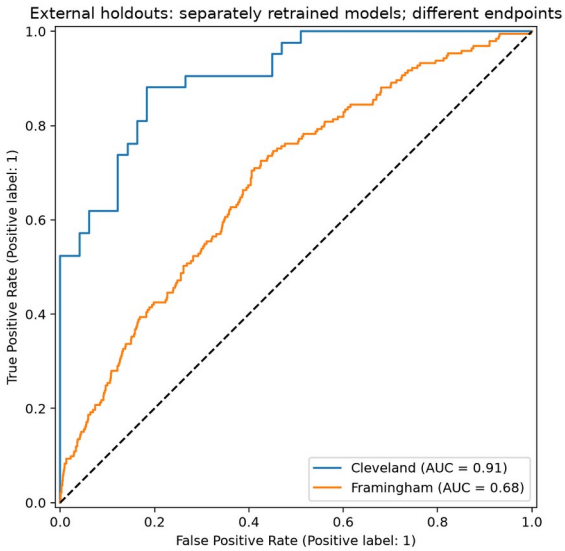


Figure 14. ROC curves on the Cleveland and Framingham holdouts.

8. Additional validation

06_additional_validation.py runs two more checks. Neither of them changes the official split or the model we selected.

80/10/10 split

Same cleaning as before, but the data is split 80/10/10 (54,822 / 6,853 / 6,853 records). We used the frozen XGBoost settings and refit the imputer on the new training part.

Table 9. Results of the 80/10/10 split check (split_80_10_10_results.csv)

Model	Split	ROC-AUC	Accuracy	Recall
Dummy	Validation	0.5000	50.52%	0.00%
Dummy	Test	0.5000	50.52%	0.00%
XGBoost	Validation	0.8138	74.33%	69.09%
XGBoost	Test	0.7976	72.86%	67.59%

The test ROC-AUC of 0.7976 is close to the 0.8028 from the main split. These are the same records as the main experiment, though, so we treat this as a sensitivity check and not as independent confirmation.

Repeated cross-validation on Cleveland

We ran five-fold stratified cross-validation ten times (50 fits per model) on the 212 Cleveland training records. The 91-record holdout was left alone. The frozen settings gave a ROC-AUC of 0.8655 ± 0.0506, accuracy of 78.97% ± 5.98% and recall of 75.92% ± 9.38%. The dummy model scored 0.5000. The ± value is the spread over 50 fold scores that overlap a lot, so it is not a confidence interval. The individual scores are in cleveland_repeated_cv_folds.csv.

9. Verification and environment

verify_pipeline.py ran with no assertion errors. It checks that the three primary sets share no predictor profiles, that id is gone, and that the saved BMI, pulse pressure and MAP columns match their formulas. It also checks that the saved imputers (and the logistic regression scaler) hold statistics from the training set only, that the stored CV means and SDs match the results table, that the test ROC-AUC can be recomputed from the saved predictions, and that calling predict does not change a fitted imputer.

Table 10. Environment the code was run in (environment.json)

Item	Setting
CPU	Intel Core i5-7360U, 2.30 GHz, 4 logical CPUs
RAM	16 GiB
OS	macOS 13.7.7
Python	3.13.1
Packages	pandas 3.0.6, numpy 2.3.5, scikit-learn 1.9.1, xgboost 3.4.1, shap 0.52.0, matplotlib 3.11.2, joblib 1.6.0
Training	CPU only, four parallel grid-search jobs

10. Limitations

- We cannot show that a patient appears in only one set. The id column is a row number, so the same person could be in both training and test. Our overlap check only catches identical measurements.
- The test set was looked at in an earlier version of the pipeline. This rerun fixes the problems we found and makes no changes based on test results, but it is still not a fresh, untouched test.
- The CV scores that picked the hyperparameters are also reported as performance numbers, so they are a little optimistic compared with nested cross-validation.
- The three datasets have different predictors and different outcomes, so their scores should not be compared directly.
- Framingham recall is very low, we never tuned the threshold, and we did not check probability calibration on any dataset.
- SHAP values describe the model. They are not causes and not medical advice.

Appendix: files produced

Table 11. Output files and what they contain

File	Contents
preprocessing_audit.json	Record counts for raw, removed, clean and each split
cv_*.csv	Full grid-search results with per-fold scores for each model
validation_results.csv	Validation metrics, CV summary and chosen parameters
selection.json	Which model won and how it was chosen
test_results.csv, test_predictions.csv	Primary test metrics and per-row probabilities
shap_importance.csv, shap_metadata.json	SHAP ranking, sampled row indices, method and units
generalization_results.csv, external_audit.json	Cleveland and Framingham results and data audits
additional_validation/	80/10/10 results and the 50 repeated-CV fold scores
fitted_hyperparameters.json	Full parameter set of every fitted model
environment.json, hardware.json	Machine and package versions